



SecOps Overload?

Use AI to reduce Secops noise with Mate Security.

Le

DATA SCIENCE

LATEST

The German Tank Problem

EDITOR'S PICKS

Estimating your chances of winning the lottery with

DEEP DIVES

Dorian Drost

Mar 6, 2024 13 min read

NEWSLETTER

WRITE FOR TDS

Submit an Article

Statistical estimates can be fascinating, can't they? When you sample a few instances from a population, you can estimate properties of that population such as the mean and variance. Likewise, under the right circumstances, you can estimate the **total size** of the population, as you do in this article.

I will use the example of drawing lottery tickets to estimate how many tickets there are in total, and hence calculate the probability of winning. More formally, this means to estimate the total size given a discrete uniform distribution. We will compare different estimates and discuss their differences and when to use them. In addition, I will point you to some other use cases where this can be used in.

Playing the lottery



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Submit an Article](#)

I'm too anxious to ride one of those, but if it pleases you...Photo by [Q](#)

Let's imagine I go to a state fair and buy some lottery. As a data scientist, I want to know the winning the main prize, of course. Let's assume single ticket that wins the main prize. So, to estimate the likelihood of winning, I need to know the total tickets N , as my chance of winning is $1/N$ then tickets). But how can I estimate that N by just tickets (which are, as I saw, all losers)?

I will make use of the fact, that the lottery tickets are in order, and I assume, that these are consecutive numbers (which means, that I assume a discrete distribution). Say I have bought some tickets and their numbers in order are $[242, 412, 823, 1429, 1702]$. What do I know about the total number of tickets now? Well, obviously there are at least 1702 tickets (as that is the highest number I have seen). That gives me a first lower bound of the number of tickets.

how accurate is it for the actual number of tickets? Just because the highest number I have drawn is 1702, that doesn't mean that there are any numbers higher than that. It is very unlikely, that I caught the lottery ticket with the highest number in my sample.

However, we can make more out of the data. Let us think as follows: If we knew the middle number of all the tickets, we could easily derive the total number from that:

number is m , then there are $m-1$ tickets below number, and there are $m-1$ tickets above that. The number of tickets would be $(m-1) + (m-1) + 1$, (the ticket of number m itself), which is equal to $2m-1$. We know that middle number m , but we can estimate it as the median of our sample. My sample above has an average of 922, which yields $2 \cdot 922 - 1 = 1843$. The calculation the estimated number of tickets is

That was quite interesting so far, as just from a few numbers, I was able to give an estimate of the total number of tickets. However, you may wonder if that is the best we can get. Let me spoil you right away: It is not.

The method we used has some drawbacks. Let us illustrate that to you with another example: Say we have [12,30,88], which leads us to $2 \cdot 43 - 1 = 85$. That formula suggests there are 85 tickets in total. But we have a ticket number 88 in our sample, so this can not be correct. There is a general problem with this method: The estimate can be lower than the highest number in the sample. In this case, the estimate has no meaning at all, as we know that the highest number in the sample is a natural number of the overall N .

[LATEST](#)

[EDITOR'S PICKS](#)

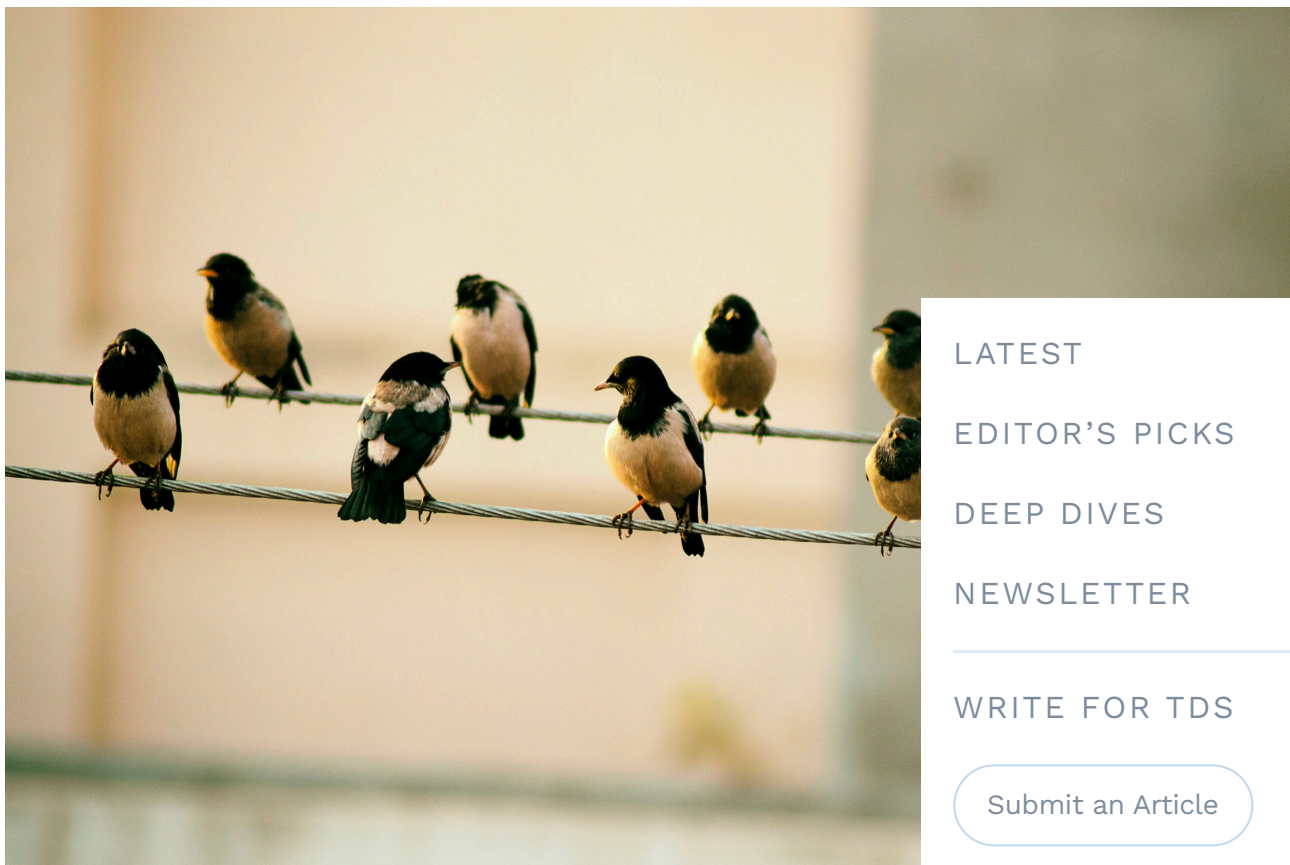
[DEEP DIVES](#)

[NEWSLETTER](#)

[WRITE FOR TDS](#)

[Submit an Article](#)

A better approach: Using even spacing



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article

These birds are quite evenly spaced on the power line, which is an important
Photo by [Ridham Nagralawala](#) on [Unsplash](#)

Okay, so what can we do? Let us think in a different way. The lottery tickets I bought have been sampled from the distribution that goes from 1 to unknown N . I know the highest number is number 1702, and I wonder what is this from being the highest ticket at all. In other words, what is the gap between 1702 and N ? If I knew that gap, I could calculate N from that. What do I know about this gap? Well, I have reason to assume that this gap is on average as big as all the other gaps between two consecutive samples. The gap between the first and the second ticket, on average, be as big as the gap between the second and the third ticket, and so on. There is no reason why this gap should be bigger or smaller than the others, except for random variation, of course. I sampled my lottery tickets from a range of 1 to N , so they should be evenly spaced on the range of 1 to N .

ticket numbers. On average, the numbers in the range of 0 to N would look like birds on a power line, all having the same gap between them.

That means I expect $N-1702$ to equal the average of all the other gaps. The other gaps are $242-0=242$, $412-242=170$, $823-412=411$, $1429-823=606$, $1702-1429=273$, which gives the average 340

Hence I estimate N to be $1702+340=2042$. In short, the formula is denoted by the following formula:

$$N = x + \frac{x - k}{k}$$

Here x is the biggest number observed (1702, in our case), k is the number of samples (5, in our case). This form of calculating the average as we just did.

Let's do a simulation

We just saw two estimates of the total number of tickets. First, we calculated $2 \cdot m - 1$, which gave us 1843. The more sophisticated approach of $x + (x - k) / k$ gave us 2042. I wonder which estimation is more correct. The chances of winning the lottery $1/1843$ or $1/2042$.

To show some properties of the estimates we just saw, I did a simulation. I drew samples of different sizes k from a uniform distribution, where the highest number is 2000, and repeated this a few hundred times each. Hence we would expect the estimates also return 2000, at least on average. The outcome of the simulation:

[LATEST](#)

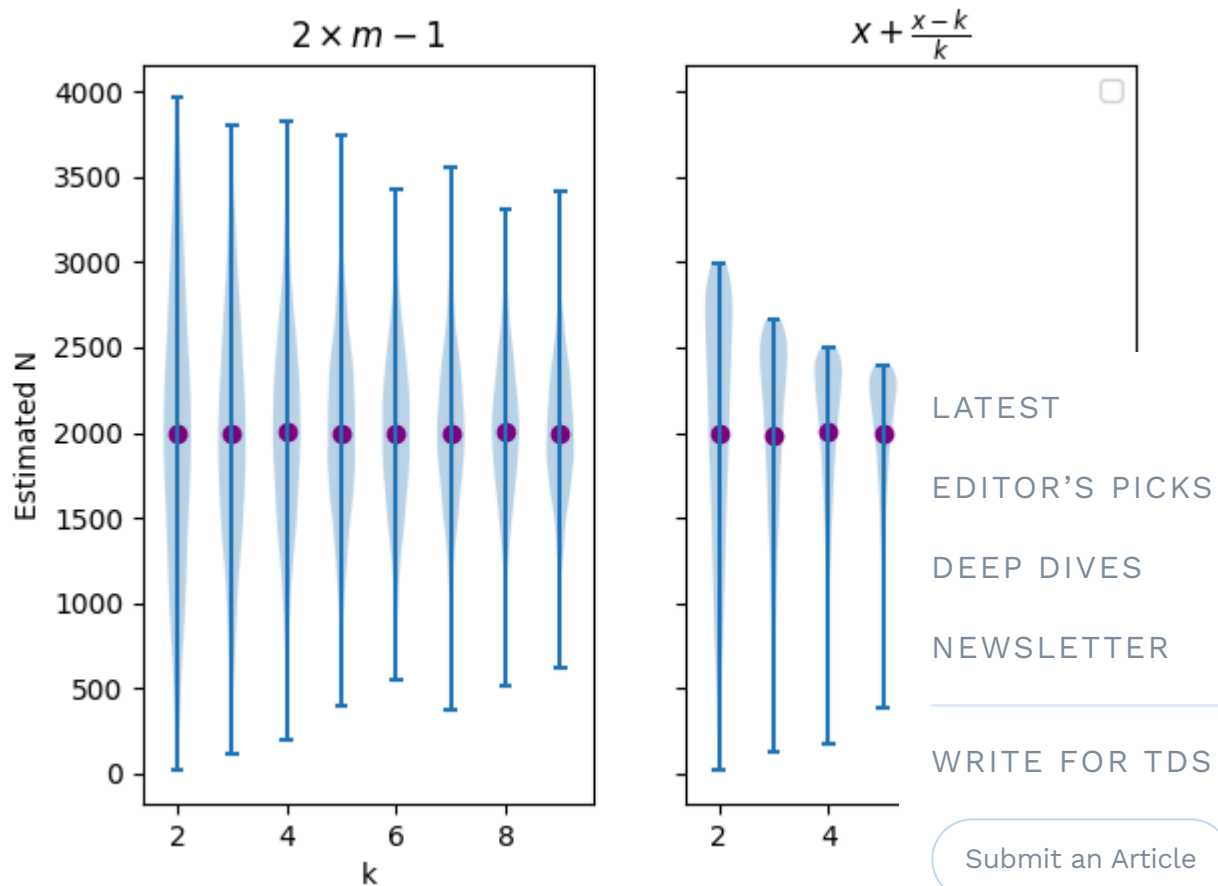
[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[NEWSLETTER](#)

[WRITE FOR TDS](#)

[Submit an Article](#)



Probability densities of the different estimates for varying k . Note that the gray dot represents the true value of the parameter being estimated.

What do we see here? On the x-axis, we see the number of samples we take. For each k , we see the distribution of the estimates based on a few hundred simulations. The dark point is the true value of the parameter, which is always k . That is a very interesting point: Both estimates converge to the correct value if they are repeated an infinite number of times.

However, besides the common average, the distributions are quite different. We see that the formula $2 \times m - 1$ has a much larger variance than the other formula. The variance decreases as k increases, but this decrease is not as pronounced as for the other formula. This is just a simulation and is still subject to random noise.

random influences. However, it is quite understandable and expected: The more samples I take, the more precise is my estimation. That is a very common property of statistical estimates.

We also see that the deviations are symmetrical, i.e. underestimating the real value is as likely as overestimating it. For the second approach, this symmetry does not hold. Most of the density is above the real mean, the larger outliers below. How does that come? Let's compute that estimate. We took the biggest number in our sample and added the average gap size to that. The biggest number in our sample can only be as big as the number in total (the N that we want to estimate). If we add the average gap size to N , but we can't get that with our estimate. In the other direction, the estimate can be very low. If we are unlucky, we could draw $[1,2,3,4,5]$, in which case the biggest number in our sample is very far away from the actual N . That is why larger deviations are possible in underestimating the real value than in overestimating it.

Which is better?

From what we just saw, which estimate is better? The one that gives the correct value on average. However, the estimate $(k+1)/k$ has lower variance, and that is a big advantage. That you are closer to the real value more often is not as important as demonstrating that to you. In the following, you will see density plots of the two estimates for a sample of size $N=10$.

[LATEST](#)

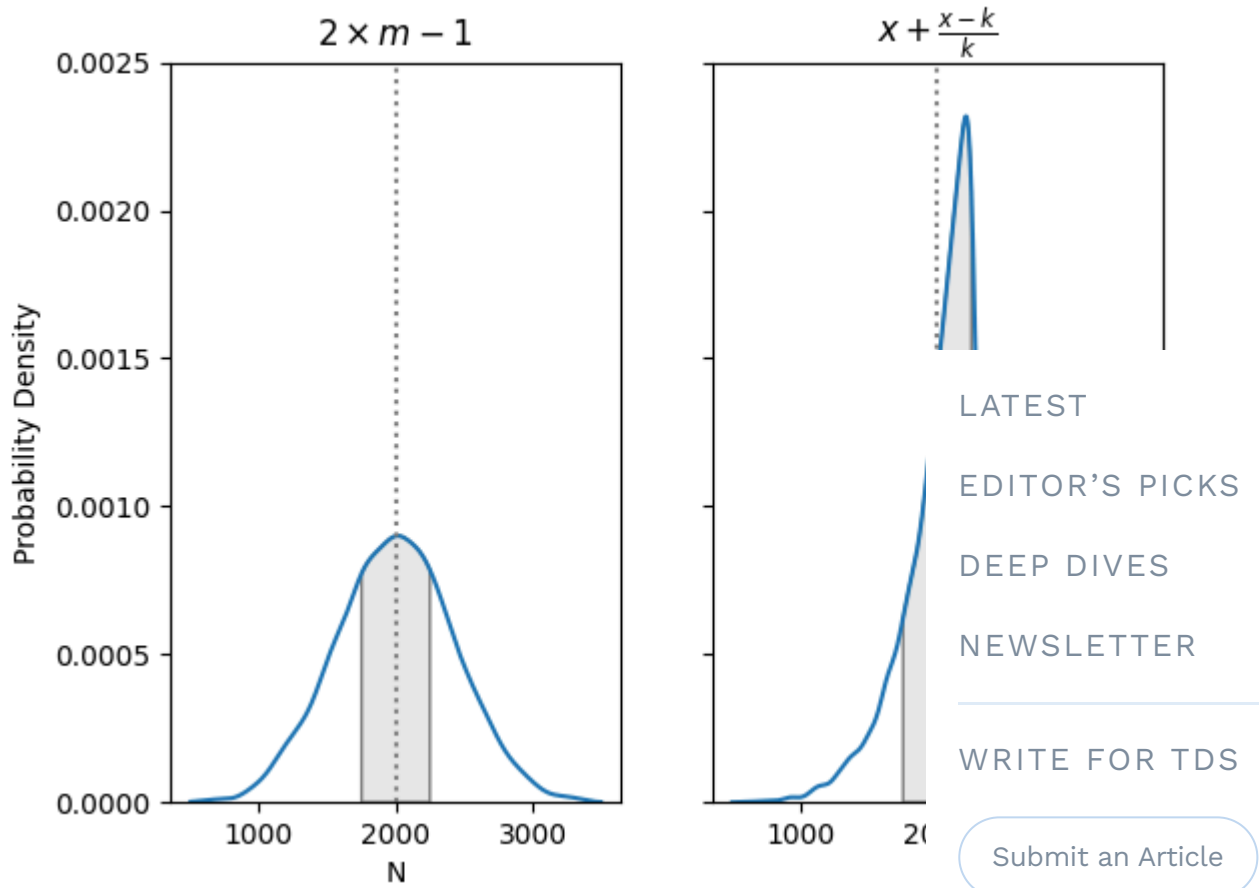
[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[NEWSLETTER](#)

[WRITE FOR TDS](#)

[Submit an Article](#)



Probability densities for the two estimates for $k=5$. The colored shape under the curves is from $N=1750$ to $N=2250$. Image by author.

I highlighted the point $N=2000$ (the real value of the line). First of all, we still see the symmetry that we saw before already. In the left plot, the density is distributed symmetrically around $N=2000$, but in the right plot, it is skewed to the right and has a longer tail to the left. Notice the grey area under the curves in both plots. In both plots, the grey area is from $N=1750$ to $N=2250$. However, in the left plot, this area accounts for 42% of the total area under the curve. In the right plot, it accounts for 73%. In other words, you have a 42% chance that your estimate is **not** more than 250 points in either direction. In the right plot, this chance is 73%. That means, you are much more likely to be close to the real value. However, you are more likely to overestimate than underestimate.

I can tell you, that $x \pm (x-k)/k$ is the so-called **uniformly minimum variance unbiased estimator**, i.e. it is the estimator with the smallest variance. You won't find any estimate with lower variance, so this is the best you can use, in general.

Use cases



Make love, not war ❤️ . Photo by [Marco Xu](#) on [Unsplash](#)

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Submit an Article](#)

We just saw how to estimate the total number pool, if these elements are indicated by consecutive numbers. Formally, this is a discrete uniform distribution commonly known as the **German tank problem**. During World War, the Allies used this approach to estimate the total number of tanks the German forces had, just by using the serial numbers of the tanks they had destroyed or captured so far.

We can now think of more examples where we can apply this approach. Some are:

- You can estimate how many instances of a product have been produced if they are labeled with a running serial number.
- You can estimate the number of users or customers if you are able to sample some of their IDs.
- You can estimate how many students are (or have been) at your university if you sample students' matriculation numbers (given that the university has not changed matriculation numbers again after reaching the maximum).

However, be aware that some requirements need to be met to use that approach. The most important one is that you draw your samples randomly and independently. If you ask your friends, who have all enrolled in the same year, their matriculation numbers, they won't be even across the whole range of matriculation numbers but will be concentrated. Likewise, if you buy articles with running numbers, you need to make sure, that this store got these articles in random fashion. If it was delivered with the product IDs 1000 to 1050, you don't draw randomly from the whole range.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article

Conclusion

We just saw different ways of estimating the total number of instances in a pool under discrete uniform distribution. Both estimates give the same expected value if the sample is drawn randomly, but they differ in terms of their variance, with one being much smaller than the other. This is interesting because neither is wrong or right. Both are backed by reasonable mathematical considerations and estimate the real population size (in frequentist statistical terms).

I now know that my chance of winning the state fair lottery is estimated to be $1/2042 = 0.041\%$ (or 0.24% with the 5 tickets I bought). Maybe I should rather invest my money in cotton candy; that would be a save win.

References & Literature

Mathematical background on the estimates discussed in this article can be found here:

- Johnson, R. W. (1994). Estimating the size of a population. *Teaching Statistics*, 16(2), 50–52.

Also feel free to check out the Wikipedia article on the German tank problem and related topics, which are quite interesting:

- https://en.wikipedia.org/wiki/German_tank_problem
- https://en.wikipedia.org/wiki/Discrete_uniform_distribution

This is the script to do the simulation and create the plots shown in the article:

```
import numpy as np
import random
from scipy.stats import gaussian_kde
import matplotlib.pyplot as plt

if __name__ == "__main__":
    N = 2000
    n_simulations = 500

    estimate_1 = lambda sample: 2 * round(np.mean(sample))
    estimate_2 = lambda sample: round(max(sample))

    estimate_1_per_k, estimate_2_per_k = [], []
    k_range = range(2, 10)
    for k in k_range:
        diffs_1, diffs_2 = [], []
```

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article

```

# sample without duplicates:
samples = [random.sample(range(N), k) for _ in range(n_sim)]
estimate_1_per_k.append([estimate_1(sample) for sample in
estimate_2_per_k.append([estimate_2(sample) for sample in

fig, axes = plt.subplots(1, 2, sharey=True, sharex=True)
axes[0].violinplot(estimate_1_per_k, positions=k_range, showext
axes[0].scatter(k_range, [np.mean(d) for d in estimate_1_per_k])
axes[1].violinplot(estimate_2_per_k, positions=k_range, showext
axes[1].scatter(k_range, [np.mean(d) for d in estimate_2_per_k])

axes[0].set_xlabel("k")
axes[1].set_xlabel("k")
axes[0].set_ylabel("Estimated N")
axes[0].set_title(r"$2\times m-1$")
axes[1].set_title(r"$x+\frac{x-k}{k}$")
plt.show()

plt.gcf().clf()
k = 5
xs = np.linspace(500, 3500, 500)

fig, axes = plt.subplots(1, 2, sharey=True)
density_1 = gaussian_kde(estimate_1_per_k)
axes[0].plot(xs, density_1(xs))
density_2 = gaussian_kde(estimate_2_per_k)
axes[1].plot(xs, density_2(xs))
axes[0].vlines(2000, ymin=0, ymax=0.003, color='m')
axes[1].vlines(2000, ymin=0, ymax=0.003, color='m')
axes[0].set_ylim(0, 0.0025)

a, b = 1750, 2250
ix = np.linspace(a, b)
verts = [(a, 0), *zip(ix, density_1(ix)),
poly = plt.Polygon(verts, facecolor='0.9',
axes[0].add_patch(poly)
print("Integral for estimate 1: ", density

verts = [(a, 0), *zip(ix, density_2(ix)),
poly = plt.Polygon(verts, facecolor='0.9',
axes[1].add_patch(poly)
print("Integral for estimate 2: ", density

```

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article

```
axs[0].set_ylabel("Probability Density")
axs[0].set_xlabel("N")
axs[1].set_xlabel("N")
axs[0].set_title(r"$2\times m-1$")
axs[1].set_title(r"$x+\frac{x-k}{k}$")

plt.show()
```

Like this article? Follow me to be notified of my

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITTEN BY

Dorian Drost

See all from Dorian Drost

WRITE FOR TDS

Submit an Article

Data Science

German Tank Problem

Getting S

Statistical Estimate

Statistics

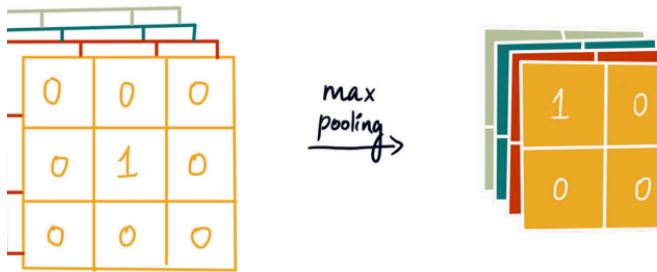
Share This Article



Towards Data Science is a community pub
your insights to reach our global audience :
the TDS Author Payment Progi

Write for TDS

Related Articles



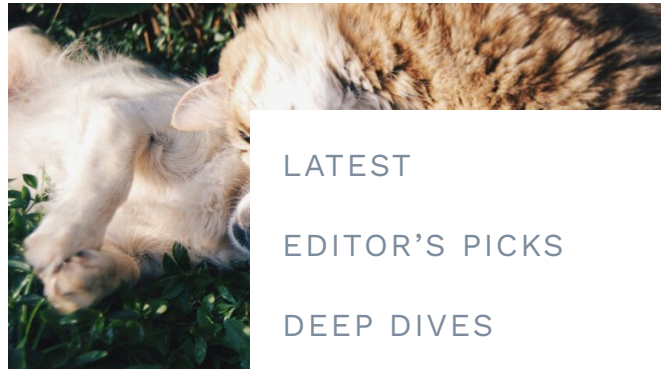
ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



ARTIFICIAL INTELLIGENCE

How to Forecast Time Series

A beginner's guide to time series reconciliation

Dr. Robert Kübler

August 20, 2024

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Submit an Article](#)



DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of...

Piero Paialunga

August 21, 2024 12 min read

DATA SCIENCE

Solving a Complex Scheduling Problem with Quantum Annealing

Solving the rescheduling problem with Wave's hybrid classical-quantum model (CQM)

Luis Fernando PÉREZ

August 20, 2024



DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read



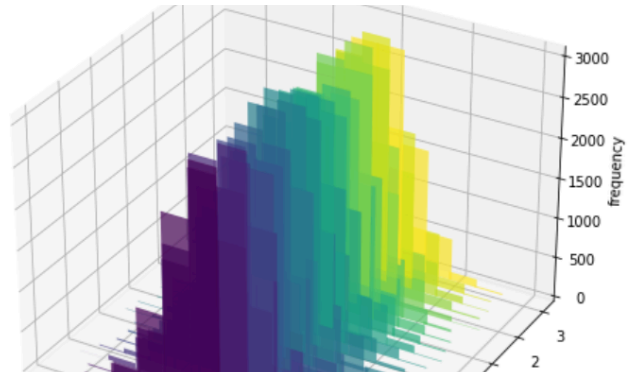
DATA SCIENCE

Our Columns

Columns on TDS are carefully curated collections of posts on a particular idea or category...

TDS Editors

November 14, 2020 4 min read



DATA SCIENCE

Must-Know Bivariate No Explained

Derivation and powerful conce

Luigi Battistoni

August 14, 2024

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article



Your home for data science and AI. The world’s leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2026

Subscribe to Our Newsletter

WRITE FOR TDS · ABOUT · ADVERTISE · PRIVACY POLICY

LATEST

EDITOR’S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Submit an Article